មេរៀនទី២ ROUTE AND CONTROLLER

ស្វែងយល់ពីគោលការណ៍ស្នូលនៃ Laravel Routing និង Controllers សម្រាប់ការអភិវឌ្ឍន៍កម្មវិធី Web

មេរៀននេះនឹងសិក្សាអំពី

01

MVC Pattern

ស្វែងយល់ពីរចនាសម្ព័ន្ធ Model-View-Controller

02

Routing

ការនាំផ្លូវនិងគ្រប់គ្រង HTTP Requests

03

Controller

ការរៀបចំ Logic និងដំណើរការទិន្នន័យ

04

View

ការបង្ហាញទិន្នន័យទៅកាន់អ្នកប្រើប្រាស់

១. អ្វីទៅជា MVC?

MVC គឺជា Software Design Pattern ដែលបំបែកកម្មវិធីជា៣ផ្នែកសំខាន់ ដើម្បីសម្រួលដល់ការអភិវឌ្ឍន៍និងថែទាំ។

Model
គ្រប់គ្រងទិន្នន័យ និងតក្កវិជ្ជា

View
បង្ហាញទិន្នន័យ

Controller
ត្រួតពិនិត្យចរន្ត

២. អ្វីទៅជា HTTP Verbs?

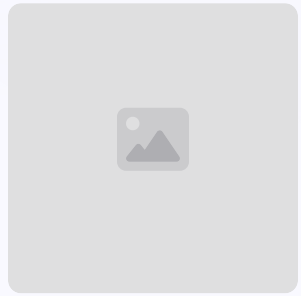
HTTP Verbs គឺជាវិធីសាស្ត្រដែលប្រើប្រាស់ក្នុងការទំនាក់ទំនងរវាង Client និង Server។

GET

ស្នើសុំទិន្នន័យពី Server ដោយមិនផ្លាស់ប្តូរអ្វីទាំងអស់

POST

បញ្ជូនទិន្នន័យថ្មីទៅ Server ដើម្បីបង្កើត



PUT

ធ្វើបច្ចុប្បន្នភាពទិន្នន័យដែលមានស្រាប់

DELETE

លុបទិន្នន័យចេញពី Server

៣. អ្វីទៅជា Laravel's Route?

Route គឺជាផ្លូវដែលកំណត់ទិសដៅពី URL ទៅកាន់ការដំណើរការជាក់លាក់មួយនៅក្នុងកម្មវិធី។ វាជាស្នូលនៃការគ្រប់គ្រង HTTP Requests ក្នុង Laravel Framework។

៣.១. អំពី Routing Template

ធាតុផ្សំនៃ Route

Route Template មានធាតុផ្សំសំខាន់ពីរប្រការ ដែលជួយកំណត់រចនាសម្ព័ន្ធនៃ URL។

- Static Route Segment - ផ្នែកថេរដែលមិនផ្លាស់ប្តូរ
- Route Value - តម្លៃអថេរដែលទទួលយកពីអ្នកប្រើប្រាស់

ឧទាហរណ៍

```
/products/{id}
```

products = Static Segment
{id} = Dynamic Parameter

៣.២. ការបង្កើត Route

Route ដើរតួនាទីជាអ្នកនាំផ្លូវទៅកាន់ទីតាំងជាក់លាក់នៅក្នុងប្រព័ន្ធ។ Laravel ផ្តល់ឯកសារពីរសម្រាប់គ្រប់គ្រង Routes៖

routes/web.php

សម្រាប់ Web Applications ដែលត្រូវការ Session និង CSRF Protection

routes/api.php

សម្រាប់ Web APIs ដែលមិនត្រូវការ Session ហើយប្រើ Token Authentication

៣.៣. មូលដ្ឋានគ្រឹះនៃ Route

Method

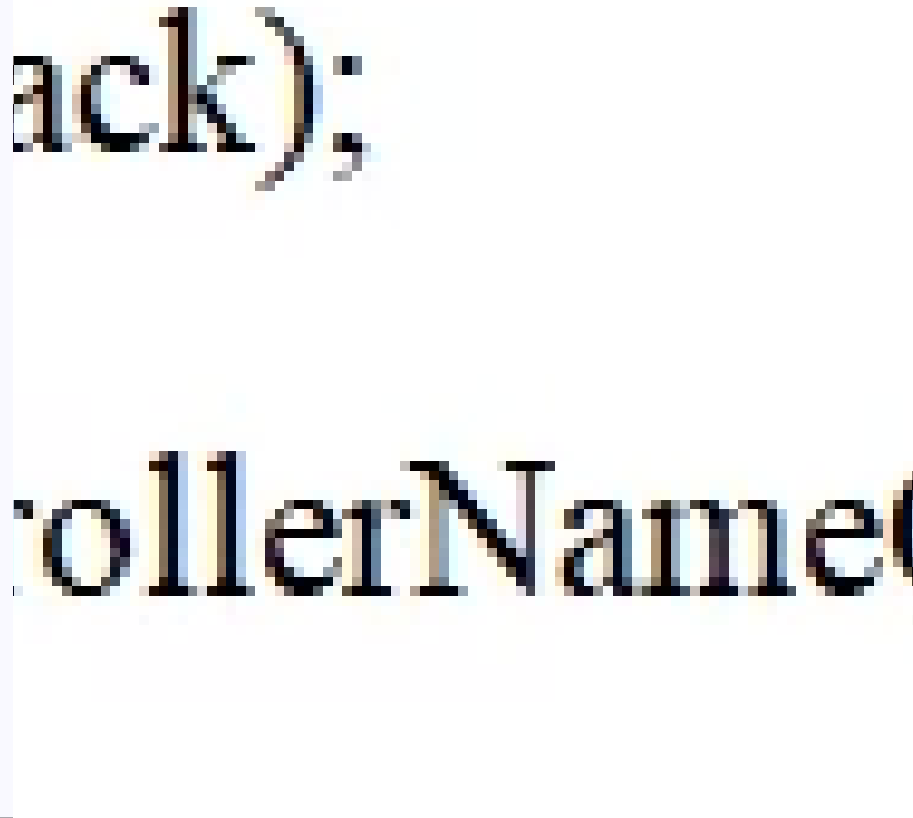
HTTP Verb ដូចជា GET, POST, PUT, DELETE ដែលកំណត់ប្រភេទនៃ Request

$url

URL Pattern ឬ Endpoint ដែលអ្នកប្រើប្រាស់ស្នើសុំ បែងចែកដោយសញ្ញា (/)

$callback

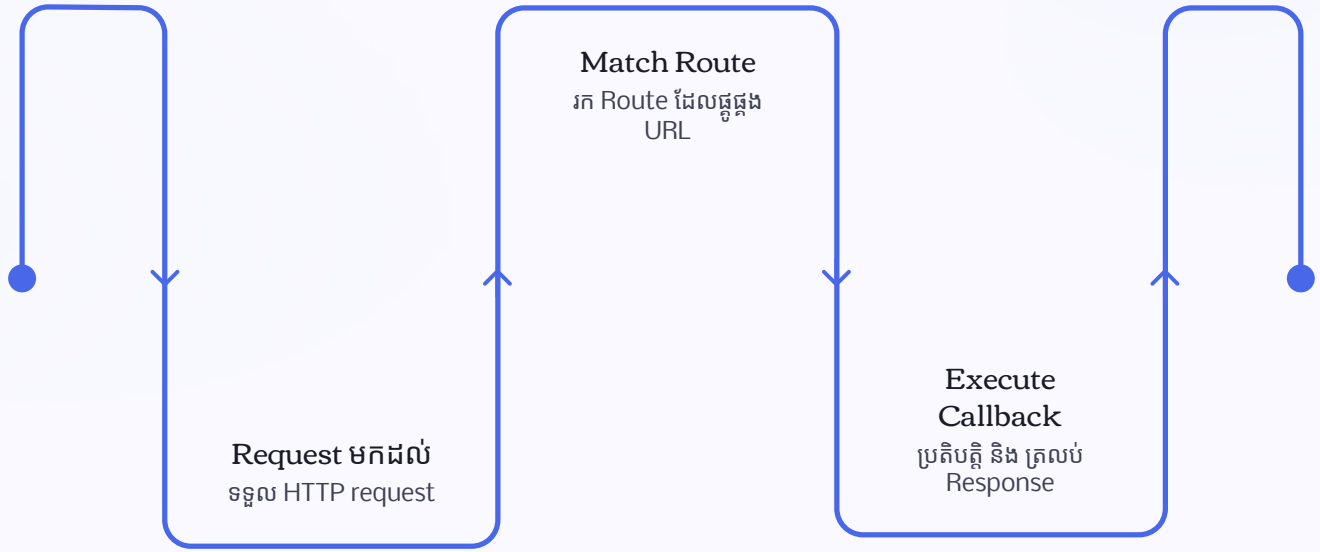
Function ដែលត្រូវប្រតិបត្តិនៅពេលទទួល Request និងបង្កើត Response



៣.៤. Route's Verb

Laravel គាំទ្រ HTTP Verbs គ្រប់ប្រភេទសម្រាប់ការគ្រប់គ្រង Routes ប្រកបដោយប្រសិទ្ធភាព។ ការជ្រើសរើស Verb ត្រឹមត្រូវជួយធានាបាននូវ RESTful Architecture ល្អ។

៣.៥. Route Handling



ដំណើរការនៃ Route

នៅពេល Request មកដល់ Laravel នឹងស្វែងរក Route ដែលត្រូវគ្នា។ បន្ទាប់មក Callback Function នឹងត្រូវប្រតិបត្តិហើយបង្កើត Response ត្រលប់មកវិញ។

៣.៦. Route Parameters

ពេលខ្លះយើងត្រូវការចាប់យកតម្លៃពី URL ដើម្បីប្រើប្រាស់ក្នុង Application។ Route Parameters អនុញ្ញាតឱ្យយើងធ្វើដូច្នេះបាន។

Required Parameters

តម្រូវឱ្យមានតម្លៃជាចាំបាច់

Optional Parameters

អាចមានឬមិនមានក៏បាន

ក. Required Parameters

រចនាសម្ព័ន្ធ

```
Route::HttpVerb("section/{parameter}",
  function($parameter) {
    // ដំណើរការរបស់អ្នកនៅទីនេះ
  }
);
```

Route Parameters ត្រូវតែត្រូវបានកំណត់នៅក្នុងសញ្ញា {parameter} នៅក្នុង Route URI។

តម្លៃនេះត្រូវតែមានជានិច្ច ហើយនឹងត្រូវបញ្ជូនទៅ Callback Function។

ឧទាហរណ៍៖

```
Route::get('/user/{id}', function($id) {
 return "User ID: " . $id;
});
```

ខ. Optional Parameters

រចនាសម្ព័ន្ធ

```
Route::HttpVerb("section/{parameter?}",
  function($parameter='default value') {
    // ដំណើរការរបស់អ្នកនៅទីនេះ
  }
);
```

Optional Parameters ត្រូវបានកំណត់ដោយសញ្ញា {parameter?} ហើយត្រូវមានតម្លៃលំនាំដើម។

ឧទាហរណ៍៖

```
Route::get('/user/{name?}', function($name='Guest') {
  return "Hello " . $name;
});
```

អាច Call បាន៖ /user/John ឬ /user

៣.៧. ដាក់ឈ្មោះ Route

ការដាក់ឈ្មោះ Route ធ្វើឱ្យងាយស្រួលក្នុងការយោងទៅកាន់ Route នោះនៅកន្លែងផ្សេងក្នុង Application។

```
Route::get('/user/profile', [ProfileController::class, 'show'])
    ->name('profile');
```

បន្ទាប់មកអ្នកអាចប្រើ៖ `route('profile')` ដើម្បីបង្កើត URL

៣.៨. Route Groups

Route Groups អនុញ្ញាតឱ្យយើងដាក់ Routes ដែលមានលក្ខណៈដូចគ្នាជាក្រុម និងចែករំលែកលក្ខណៈរួមគ្នា។

Middleware

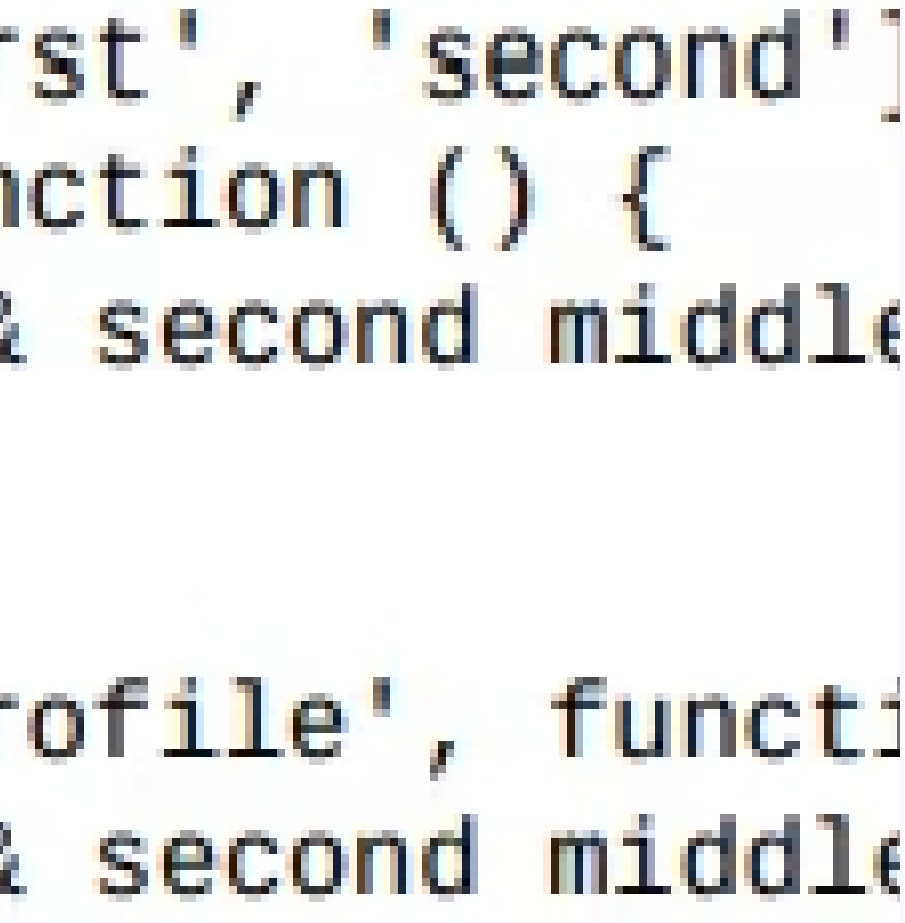
ចែករំលែក Middleware ដូចគ្នាសម្រាប់ការពារ និងត្រួតពិនិត្យ

Route Path Prefixes

បន្ថែម Prefix រួមគ្នាសម្រាប់ URL Paths

Name Prefix

បន្ថែម Prefix សម្រាប់ឈ្មោះ Routes ទាំងអស់



ក. Route Middleware

Middleware ត្រូវបានប្រតិបត្តិតាមលំដាប់ដែលបានរាយក្នុង Array។ វាជួយក្នុងការត្រួតពិនិត្យ Authentication, Authorization និងការដំណើរការផ្សេងៗមុនពេល Request ទៅដល់ Controller។

ខ. Path Route Prefix

មុខងារ prefix()

```
Route::prefix('admin')->group(function () {
    Route::get('/users', function () {
        // URL: /admin/users
    });
    Route::get('/posts', function () {
        // URL: /admin/posts
    });
});
```

Prefix() Method អនុញ្ញាតឱ្យយើងបន្ថែមផ្នែកខាងមុខរួមគ្នាសម្រាប់ URLs ទាំងអស់នៅក្នុងក្រុម។

វាជួយធ្វើឱ្យកូដមានភាពច្បាស់លាស់និងងាយស្រួលក្នុងការថែទាំ។

គ. Named Prefix

មុខងារ name()

```
Route::name('admin.')->group(function () {
    Route::get('/users', function () {
        // Route name: admin.users
    })->name('users');

    Route::get('/posts', function () {
        // Route name: admin.posts
    })->name('posts');
});
```

Name() Method ជួយបន្ថែម Prefix សម្រាប់ឈ្មោះ Routes ទាំងអស់នៅក្នុងក្រុម។

វាងាយស្រួលក្នុងការគ្រប់គ្រងនិងយោងទៅកាន់ Routes ក្នុង Application ធំៗ។

៣.៩. Route Fallback

ការចាប់យក Request ដែលមិនត្រូវគ្នា

Fallback Route ត្រូវបានប្រើនៅពេល Request មិនត្រូវនឹង Route ណាមួយទេ។ វាត្រូវតែកំណត់នៅចុងបំផុតនៃឯកសារ Routes។

```
Route::fallback(function () {
 return response()-
>view('errors.404', [], 404);
});
```

៣.១០. Route Constraints

Route Constraints កំណត់ទម្រង់ត្រឹមត្រូវនៃ Route Parameters ដោយប្រើ Regular Expression។

whereNumber()

```
Route::get('/user/{id}', function ($id) {
    //
})->whereNumber('id');
```

ទទួលតែលេខប៉ុណ្ណោះ

whereAlpha()

```
Route::get('/user/{name}', function ($name) {
    //
})->whereAlpha('name');
```

ទទួលតែអក្សរប៉ុណ្ណោះ

whereAlphaNumeric()

```
Route::get('/post/{slug}', function ($slug) {
    //
})->whereAlphaNumeric('slug');
```

ទទួលអក្សរនិងលេខ

៤. View

View គឺជា User Interface ដែលទទួលខុសត្រូវក្នុងការបង្ហាញទិន្នន័យពី Model ទៅកាន់អ្នកប្រើប្រាស់និងទទួលយក Input ពីពួកគេ។

> Views ទាំងអស់ត្រូវរក្សាទុកនៅក្នុងថត `resources/views` ហើយមានកន្ទុយ `*.blade.php`

៤.១. ការបង្កើត View

ដោយដៃ

បង្កើតឯកសារ .blade.php ដោយផ្ទាល់នៅក្នុងថត resources/views

```
resources/views/welcome.blade.php
```

តាមបញ្ជា Artisan

ប្រើបញ្ជា Artisan ដើម្បីបង្កើត View យ៉ាងលឿន

```
php artisan make:view welcome
```

៤.២. Route's Views

បង្កើត Route ទៅ View

Laravel អនុញ្ញាតឱ្យយើងបង្កើត Route ដែលបង្ហាញ View ដោយផ្ទាល់ ដោយមិនចាំបាច់ប្រើ Controller។

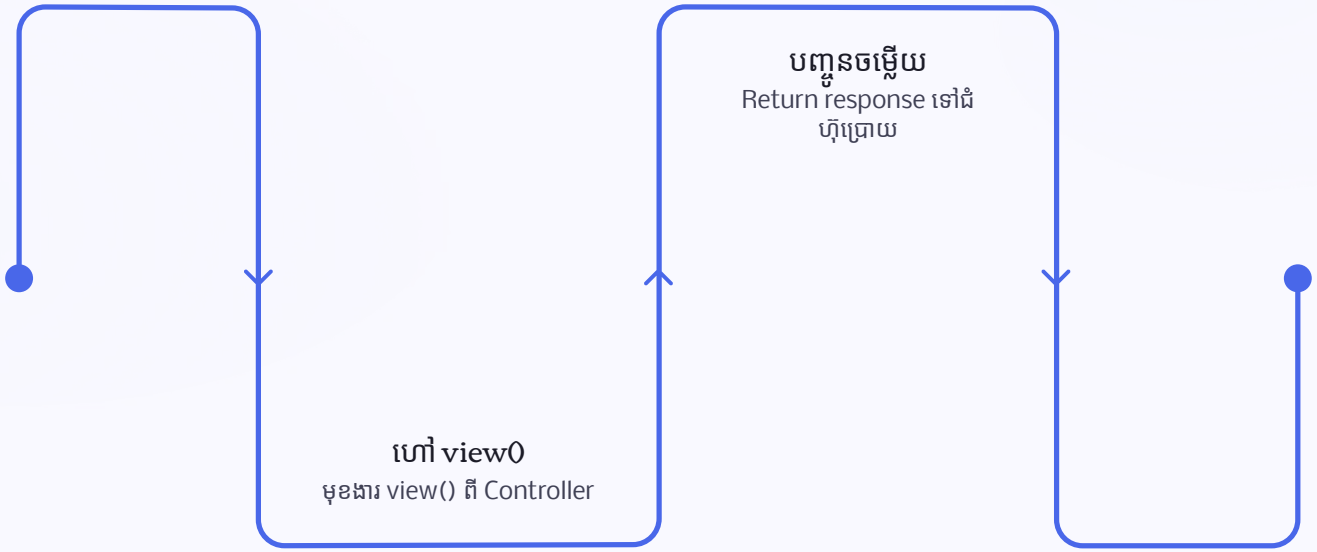
```
Route::view('/welcome', 'welcome');
```

ជាមួយទិន្នន័យ

```
Route::view('/welcome', 'welcome', [
    'name' => 'Laravel'
]);
```

អ្នកអាចបញ្ជូនទិន្នន័យជា Array ទីបី

៤.៣. បង្ហាញ Views



ការប្រើ view() Helper

មុខងារជំនួយ view() ត្រូវបានប្រើដើម្បីបង្ហាញ View ពី Controller ឬ Route។

```
return view('welcome');
```

Laravel នឹងស្វែងរកឯកសារ welcome.blade.php នៅក្នុងថត resources/views

៤.៤. បញ្ជូនទិន្នន័យទៅ Views

មានវិធីសាស្ត្រច្រើនក្នុងការបញ្ជូនទិន្នន័យទៅ View។

Array Method

```
return view('user', [
    'name' => 'John',
    'age' => 25
]);
```

Compact Method

```
$name = 'John';
$age = 25;
return view('user', compact('name', 'age'));
```

With Method

```
return view('user')
    ->with('name', 'John')
    ->with('age', 25);
```

With Array

```
return view('user')->with([
 'name' => 'John',
 'age' => 25
]);
```

៥. Controller

Controllers គឺជា Classes ដែលរៀបចំនិងប្រមូលផ្តុំ Logic របស់ Routes មួយឬច្រើនទុកនៅកន្លែងតែមួយ។ វាជួយធ្វើឱ្យកូដមានរចនាសម្ព័ន្ធល្អនិងងាយស្រួលថែទាំ។

> Controllers ទាំងអស់ត្រូវរក្សាទុកនៅក្នុងថត `app/Http/Controllers`

៥.១. ការបង្កើត Controller

ឈ្មោះត្រូវផ្តើមដោយអក្សរធំ

ឧទាហរណ៍: UserController, PostController

ឈ្មោះត្រូវបញ្ចប់ដោយ Controller

ធានាបាននូវការកំណត់អត្តសញ្ញាណយ៉ាងច្បាស់

កុំប្រើសញ្ញាពិសេស

មិនប្រើ $, &, *, #, @ ក្នុងឈ្មោះ

៥.២. ការកំណត់ Route ជាមួយ Controller

រចនាសម្ព័ន្ធ

- **ControllerName**: ឈ្មោះ Controller ដែលបានបង្កើត
- **methodName**: ឈ្មោះ Method នៅក្នុង Controller

ឧទាហរណ៍

```
Route::get('/users', [UserController::class, 'index']);
```

បង្ហាញបញ្ជីអ្នកប្រើប្រាស់ទាំងអស់

ឧទាហរណ៍ Controller

```
namespace App\Http\Controllers;

use Illuminate\Http\Request;

class UserController extends Controller
{
 public function index()
 {
 $users = User::all();
 return view('users.index', compact('users'));
 }

 public function show($id)
 {
 $user = User::findOrFail($id);
 return view('users.show', compact('user'));
 }
}
```

៥.៣. ទទួលទិន្នន័យពីអ្នកប្រើ

request() Helper

```
$name = request('name');
$email = request('email');
```

Dependency Injection

```
public function store(Request $request)
{
    $name = $request->name;
    $email = $request->email;
}
```

Dependency Injection គឺជាវិធីល្អប្រសើរជាងព្រោះវាងាយស្រួលក្នុងការធ្វើ Testing និងមានភាពច្បាស់លាស់។

៥.៤. Controller Resource

Resource Controller គឺជា Controller ដែល Laravel រៀបចំអោយសម្រាប់គ្រប់គ្រងសកម្មភាព CRUD ជាស្តង់ដារ។ វាមាន Methods ជាស្រេចសម្រាប់ការបង្កើត អាន ធ្វើបច្ចុប្បន្នភាព និងលុប។

ក. បញ្ជាបង្កើត Resource Controller

ជម្រើសទី១

```
php artisan make:controller UserController --resource
```

ប្រើ --resource flag ពេញលេញ

ជម្រើសទី២

```
php artisan make:controller UserController -r
```

ប្រើទម្រង់កាត់ -r

ទាំងពីរបញ្ជានេះបង្កើត Controller ដូចគ្នា ជាមួយ Methods ស្តង់ដារសម្រាប់ CRUD Operations។

ខ. ចុះបញ្ជី Resourceful Route

Route តែមួយ

```
Route::resource('posts', PostController::class);
```

បង្កើត Routes ទាំងអស់សម្រាប់ CRUD Operations

Routes ច្រើន

```
Route::resources([
    'posts' => PostController::class,
    'comments' => CommentController::class
]);
```

ចុះបញ្ជី Resources ច្រើនក្នុងពេលតែមួយ

គ. រាយបញ្ជី Routes

បញ្ជា Artisan

```
php artisan route:list --name=posts
```

បង្ហាញបញ្ជី Routes ទាំងអស់ដែលទាក់ទងនឹង Resource ជាក់លាក់

ទម្រង់កាត់

```
php artisan r:l --name=posts
```

ឃ. មុខងារ Methods របស់ Resource Controller

index()	បង្ហាញបញ្ជី Resources ទាំងអស់
create()	បង្ហាញ Form សម្រាប់បង្កើតថ្មី
store()	រក្សាទុកទិន្នន័យថ្មីក្នុង Database
show($id)	បង្ហាញ Resource តែមួយជាក់លាក់
edit($id)	បង្ហាញ Form សម្រាប់កែប្រែ
update($id)	ធ្វើបច្ចុប្បន្នភាពទិន្នន័យ
destroy($id)	លុប Resource ចេញ

ង. Resource Parameters

ប្តូរឈ្មោះ Parameter

```
Route::resource('posts', PostController::class)
    ->parameter('posts', 'article');
```

URL នឹងប្រើ {article} ជំនួស {post}

ប្តូរច្រើន Parameters

```
Route::resource('posts', PostController::class)
 ->parameters([
 'posts' => 'article'
 ]);
```

៥.៥. Route Model Binding

Route Model Binding ជួយធ្វើឱ្យងាយស្រួលក្នុងការយក Model Instances ពី Route Parameters។ Laravel នឹងស្វែងរក Model ដោយស្វ័យប្រវត្តិហើយបញ្ជូនទៅ Controller។

ក. Implicit Route Model Binding

Route Definition

```
Route::get('/users/{user}', [UserController::class, 'show']);
```

Controller Method

```
public function show(User $user)
{
    return view('user.profile', compact('user'));
}
```

Laravel ស្វែងរក User ដោយស្វ័យប្រវត្តិតាម ID

ខ. Custom Route Model Binding

ក្នុង Model

```
public function getRouteKeyName()
{
    return 'slug';
}
```

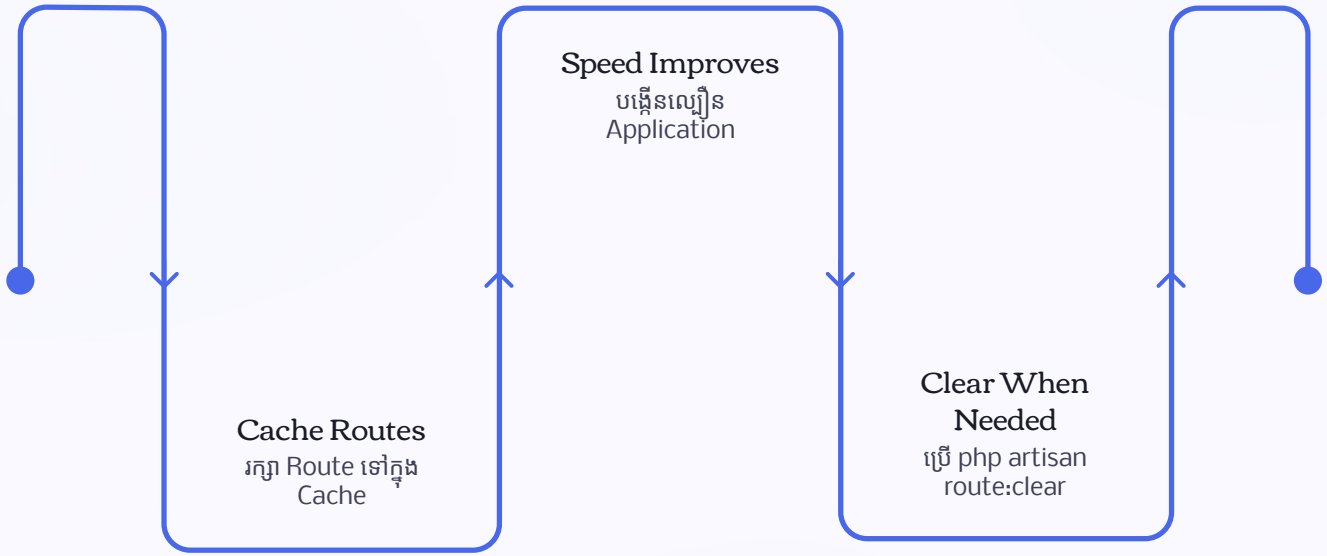
ប្តូរពី ID ទៅ Slug

ក្នុង RouteServiceProvider

```
Route::bind('user', function ($value) {
    return User::where('name', $value)->firstOrFail();
});
```

Custom Logic ពេញលេញ

៥.៦. Route Caching



បង្កើន Performance

Route Caching ជួយបង្កើនល្បឿននៃ Application ដោយ Cache Routes ទាំងអស់។

```
php artisan route:cache
```

សម្អាត Cache

```
php artisan route:clear
```

> មិនដំណើរការជាមួយ Closure Routes

៦. Form Method Spoofing

HTML Forms គាំទ្រតែ GET និង POST ប៉ុណ្ណោះ។ ដើម្បីប្រើ PUT, PATCH ឬ DELETE យើងត្រូវការ Method Spoofing។

៦.១. HTTP Verbs ក្នុង Laravel

Laravel គាំទ្រ HTTP Verbs គ្រប់ប្រភេទតាមរយៈ Route Methods ផ្សេងៗ។

Route::get()

សម្រាប់ GET Requests

Route::post()

សម្រាប់ POST Requests

Route::any()

ទទួល Verbs ទាំងអស់

Route::match()

ផ្គូផ្គង Verbs ជាក់លាក់

៦.២. HTTP Method Spoofing

HTML Form

```
<form action="/user/1" method="POST">
    @csrf
    @method('PUT')
    <input type="text" name="name">
    <button type="submit">Update</button>
</form>
```

ប្រើ `@method('PUT')` Directive ដើម្បីបញ្ជាក់ HTTP Method ដែលពិត។

Laravel នឹងយក Hidden Field នេះមកប្រើដើម្បីកំណត់ Request Method ពិតប្រាកដ។

៧. CSRF Protection

CSRF (Cross-Site Request Forgery) Protection ការពារពី Attacks ដែលព្យាយាមបញ្ជូន Requests មិនស្របច្បាប់តាមឈ្មោះអ្នកប្រើប្រាស់។

```
<form method="POST" action="/profile">
    @csrf
    ...
</form>
```

ត្រូវតែបន្ថែម @csrf ក្នុង Forms ទាំងអស់ដែលប្រើ POST, PUT, PATCH ឬ DELETE

៨. ការប្តូរទិសដៅ (Redirect)

Redirects ជួយនាំអ្នកប្រើប្រាស់ពី URL មួយទៅ URL មួយទៀតបន្ទាប់ពីដំណើរការបញ្ចប់។

redirect()

Redirect ទៅ URL ជាក់លាក់

redirect()->route()

Redirect ទៅ Named Route

redirect()->back()

ត្រឡប់ទៅទំព័រមុន

redirect()->action()

Redirect ទៅ Controller Action

៨.១. មុខងារ Redirect ផ្សេងៗ

redirect('/home')	Redirect ទៅ Path ជាក់លាក់
redirect()->route('profile')	Redirect ទៅ Named Route
redirect()->back()	ត្រឡប់ទៅទំព័រមុន
redirect()->with('key', 'value')	Redirect ជាមួយ Flash Data
redirect()->away('https://...')	Redirect ទៅ External URL

៨.២. Aborting the Request

abort() Helper

```
if (!auth()->check()) {
    abort(403, 'Unauthorized action.');
}
```

បញ្ឈប់ Request ភ្លាមៗ

abort_if() និង abort_unless()

```
abort_if(!auth()->check(), 403);

abort_unless(auth()->check(), 403);
```

Abort មានលក្ខខណ្ឌ

៨.៣. មុខងារជំនួយ URL()

url() Helper

```
$url = url('/profile');
// http://example.com/profile

$url = url('/user', [1]);
// http://example.com/user/1
```

route() Helper

```
$url = route('profile');
// URL for named route

$url = route('user.show', ['id' => 1]);
// URL with parameters
```

៩. Custom Response

Laravel អនុញ្ញាតឱ្យយើងបង្កើត Custom Responses ប្រភេទផ្សេងៗសម្រាប់ចម្លើយតបទៅ Requests។

response()->json()	JSON Response សម្រាប់ APIs
response()->download()	ទាញយកឯកសារ
response()->file()	បង្ហាញឯកសារក្នុង Browser
response()->view()	Response ជាមួយ View

ឧទាហរណ៍ Custom Response

JSON Response

```
return response()->json([
    'name' => 'John',
    'status' => 'active'
], 200);
```

Download Response

```
return response()->download(
    storage_path('app/file.pdf'),
    'document.pdf'
);
```

Custom Headers

```
return response('Hello')
 ->header('Content-Type',
'text/plain')
 ->header('X-Custom', 'Value');
```

១០. អនុវត្តជាមួយ Basic Route (Post Category)

ការអនុវត្តជាក់ស្តែងប្រើ Query Builder សម្រាប់គ្រប់គ្រង Post Categories។

Insert

បញ្ចូល Category ថ្មី

```
DB::table('categories')->insert([
    'name' => 'Technology',
    'created_at' => now()
]);
```

Update

ធ្វើបច្ចុប្បន្នភាព Category

```
DB::table('categories')
    ->where('id', 1)
    ->update(['name' => 'Tech News']);
```

Delete

លុប Category

```
DB::table('categories')
    ->where('id', 1)
    ->delete();
```

Select

ស្វែងរក និងត្រង Categories

```
$categories = DB::table('categories')
 ->where('name', 'like', '%Tech%')
 ->get();
```

១១. អនុវត្តជាមួយ Basic Route (Post)

ការអនុវត្តពេញលេញសម្រាប់គ្រប់គ្រង Posts ដោយប្រើ Query Builder។

Insert Post

```
DB::table('posts')->insert([
    'title' => 'My First Post',
    'content' => 'Content here...',
    'category_id' => 1
]);
```

Update Post

```
DB::table('posts')
    ->where('id', 1)
    ->update(['title' => 'Updated Title']);
```

Delete Post

```
DB::table('posts')
    ->where('id', 1)
    ->delete();
```

Select Specific

```
$post = DB::table('posts')
    ->where('id', 1)
    ->first();
```

Select with Filter

```
$posts = DB::table('posts')
 ->where('category_id', 1)
 ->where('title', 'like', '%Laravel%')
 ->orderBy('created_at', 'desc')
 ->get();
```